

Database implementation

GCP Connection

```
mysql> SHOW TABLES;
+-----+
| Tables_in_databased |
+-----+
| Buildings            |
| Favorites            |
| HardReservations     |
| Rooms               |
| UserReservations     |
| Users               |
+-----+
6 rows in set (0.03 sec)
```

Data Definition Language (DDL) Commands

```
CREATE TABLE Buildings (
    BuildingId VARCHAR(7) PRIMARY KEY,
    BuildingName VARCHAR(75) NOT NULL,
    Longitude DECIMAL(11, 8),
    Latitude DECIMAL(10, 8),
    UNIQUE(Longitude, Latitude) -- Enforces uniqueness for locations
);

CREATE TABLE Rooms (
    RoomNumber VARCHAR(10) NOT NULL,
    BuildingId VARCHAR(7) NOT NULL,
    Capacity INT,
    PRIMARY KEY (RoomNumber, BuildingId),
    FOREIGN KEY (BuildingId) REFERENCES Buildings(BuildingId) ON DELETE
CASCADE
);

CREATE TABLE Users (
    UID INT PRIMARY KEY AUTO_INCREMENT,
```

```

        Email VARCHAR(50) NOT NULL UNIQUE, -- Email is unique per functional
dependency
        FullName VARCHAR(50) NOT NULL
    );

CREATE TABLE Favorites (
    UID INT NOT NULL,
    BuildingId VARCHAR(7) NOT NULL,
    PRIMARY KEY (UID, BuildingId),
    FOREIGN KEY (UID) REFERENCES Users(UID) ON DELETE CASCADE,
    FOREIGN KEY (BuildingId) REFERENCES Buildings(BuildingId) ON DELETE
CASCADE
);

CREATE TABLE UserReservations (
    ReservationId INT PRIMARY KEY AUTO_INCREMENT,
    Date VARCHAR(10) NOT NULL,
    StartTime VARCHAR(10) NOT NULL,
    EndTime VARCHAR(10) NOT NULL,
    UID INT NOT NULL,
    RoomNumber VARCHAR(10) NOT NULL,
    BuildingId VARCHAR(7) NOT NULL,
    FOREIGN KEY (UID) REFERENCES Users(UID) ON DELETE CASCADE,
    FOREIGN KEY (RoomNumber, BuildingId) REFERENCES Rooms(RoomNumber,
BuildingId) ON DELETE CASCADE
);

CREATE TABLE HardReservations (
    EventId INT PRIMARY KEY AUTO_INCREMENT,
    StartDate VARCHAR(10) NOT NULL,
    EndDate VARCHAR(10) NOT NULL,
    Repeats VARCHAR(10), -- Nullable since Repeats is optional
    StartTime VARCHAR(10) NOT NULL,
    EndTime VARCHAR(10) NOT NULL,
    EventName VARCHAR(100) NOT NULL,
    RoomNumber VARCHAR(10) NOT NULL,
    BuildingId VARCHAR(7) NOT NULL,
    FOREIGN KEY (RoomNumber, BuildingId) REFERENCES Rooms(RoomNumber,
BuildingId) ON DELETE CASCADE
);

```

Count queries for three 1000+ entry tables

```
| Tables_in_databases |
+-----+
| Buildings           |
| Favorites            |
| HardReservations    |
| Rooms               |
| UserReservations    |
| Users                |
+-----+
6 rows in set (0.03 sec)

mysql> SELECT COUNT(*) FROM Users;
+-----+
| COUNT(*) |
+-----+
|      1986 |
+-----+
1 row in set (0.02 sec)

mysql> SELECT COUNT(*) FROM UserReservations;
+-----+
| COUNT(*) |
+-----+
|      1970 |
+-----+
1 row in set (0.02 sec)

mysql> SELECT COUNT(*) FROM HardReservations;
+-----+
| COUNT(*) |
+-----+
|      8825 |
+-----+
1 row in set (0.02 sec)
```

Complex Queries

Query 1

This query identifies the number of rooms available everyday at 12 PM in a user's favorite buildings.

```
SELECT
    b.BuildingName,
    COUNT(r.RoomNumber) AS AvailableRooms
FROM
    Buildings b
JOIN
    Favorites f ON b.BuildingId = f.BuildingId
JOIN
    Rooms r ON b.BuildingId = r.BuildingId
```

```

LEFT JOIN
    HardReservations hr
    ON r.RoomNumber = hr.RoomNumber
    AND r.BuildingId = hr.BuildingId
    AND TIME(STR_TO_DATE(hr.StartTime, '%I:%i %p')) <= '12:00:00'
    AND TIME(STR_TO_DATE(hr.EndTime, '%I:%i %p')) > '12:00:00'
WHERE
    f.UID = 751 -- User ID parameter
    AND hr.RoomNumber IS NULL -- Ensures no reservation at 12 PM
GROUP BY
    b.BuildingName
ORDER BY
    AvailableRooms DESC,
    b.BuildingName;

```

Output of the query:

```

mysql> SELECT
->     b.BuildingName,
->     COUNT(r.RoomNumber) AS AvailableRooms
-> FROM
->     Buildings b
-> JOIN
->     Favorites f ON b.BuildingId = f.BuildingId
-> JOIN
->     Rooms r ON b.BuildingId = r.BuildingId
-> LEFT JOIN
->     HardReservations hr
->     ON r.RoomNumber = hr.RoomNumber
->     AND r.BuildingId = hr.BuildingId
->     AND TIME(STR_TO_DATE(hr.StartTime, '%I:%i %p')) <= '12:00:00'
->     AND TIME(STR_TO_DATE(hr.EndTime, '%I:%i %p')) > '12:00:00'
-> WHERE
->     f.UID = 751 -- User ID parameter
->     AND hr.RoomNumber IS NULL -- Ensures no reservation at 12 PM
-> GROUP BY
->     b.BuildingName
-> ORDER BY
->     AvailableRooms DESC,
->     b.BuildingName;
+-----+-----+
| BuildingName | AvailableRooms |
+-----+-----+
| Armory       | 4              |
+-----+-----+
1 row in set (0.02 sec)

```

Index	Columns indexed	Query Cost	Notes
Default	PKs and FKs	28.4	Baseline
idx_hardreservations_time	HardReservations (StartTime, EndTime)	28.4	No speedup
idx_hardreservations_time_room	HardReservations (RoomNumber, BuildingId, StartTime, EndTime)	16.6	Speedup caused by indexing RoomNumber and BuildingId, which are used to join
idx_buildname	Buildings (BuildingName)	16.6	No speedup

Default:

```
| -> Sort: AvailableRooms DESC, b.BuildingName (actual time=6.27..6.27 rows=1 loops=1)
|   -> Table scan on <temporary> (actual time=5.23..5.23 rows=1 loops=1)
|     -> Aggregate using temporary table (actual time=5.23..5.23 rows=1 loops=1)
|       -> Filter: (hr.RoomNumber is null) (cost=28.4 rows=76.4) (actual time=3.53..5.15 rows=4 loops=1)
|         -> Nested loop antijoin (cost=28.4 rows=76.4) (actual time=3.53..5.14 rows=4 loops=1)
|           -> Nested loop inner join (cost=1.63 rows=6.52) (actual time=0.151..0.175 rows=28 loops=1)
|             -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.133..0.138 rows=1 loops=1)
|               -> Covering index lookup on f using PRIMARY (UID=751) (cost=0.35 rows=1) (actual time=0.056..0.0595 rows=1 loops=1)
|               -> Single-row index lookup on b using PRIMARY (BuildingId=f.BuildingId) (cost=0.35 rows=1) (actual time=0.0753..0.0755 rows=1 loops=1)
|             -> Covering index lookup on r using idx_rooms_buid_rmb (BuildingId=f.BuildingId) (cost=0.926 rows=6.52) (actual time=0.0168..0.0337 rows=28 loops=1)
|           -> Filter: ((cast(str_to_date(hr.StartTime,'%I:%i %p') as time) <= '12:00:00') and (cast(str_to_date(hr.EndTime,'%I:%i %p') as time) > '12:00:00')) (cost=3.11 rows=11.7) (actual time=0.177..0.177 rows=0.857 loops=28)
|         -> Index lookup on hr using RoomNumber (RoomNumber=r.RoomNumber, BuildingId=f.BuildingId) (cost=3.11 rows=11.7) (actual time=0.0634..0.0707 rows=4.5 loops=28)
```

idx_hardreservations_time:

```
| -> Sort: AvailableRooms DESC, b.BuildingName (actual time=0.39..0.39 rows=1 loops=1)
|   -> Table scan on <temporary> (actual time=7.34..7.34 rows=1 loops=1)
|     -> Aggregate using temporary table (actual time=7.34..7.34 rows=1 loops=1)
|       -> Filter: (hr.RoomNumber is null) (cost=28.4 rows=76.4) (actual time=4.9..7.27 rows=4 loops=1)
|         -> Nested loop antijoin (cost=28.4 rows=76.4) (actual time=4.89..7.26 rows=4 loops=1)
|           -> Nested loop inner join (cost=1.63 rows=6.52) (actual time=0.177..0.209 rows=28 loops=1)
|             -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.0905..0.0946 rows=1 loops=1)
|               -> Covering index lookup on f using PRIMARY (UID=751) (cost=0.35 rows=1) (actual time=0.0654..0.0685 rows=1 loops=1)
|               -> Single-row index lookup on b using PRIMARY (BuildingId=f.BuildingId) (cost=0.35 rows=1) (actual time=0.0222..0.0224 rows=1 loops=1)
|             -> Covering index lookup on r using idx_rooms_buid_rmb (BuildingId=f.BuildingId) (cost=0.926 rows=6.52) (actual time=0.0858..0.109 rows=28 loops=1)
|           -> Filter: ((cast(str_to_date(hr.StartTime,'%I:%i %p') as time) <= '12:00:00') and (cast(str_to_date(hr.EndTime,'%I:%i %p') as time) > '12:00:00')) (cost=3.11 rows=11.7) (actual time=0.252..0.252 rows=0.857 loops=28)
|         -> Index lookup on hr using RoomNumber (RoomNumber=r.RoomNumber, BuildingId=f.BuildingId) (cost=3.11 rows=11.7) (actual time=0.105..0.11 rows=4.5 loops=28)
```

idx_hardreservations_time_room:

```
| -> Sort: AvailableRooms DESC, b.BuildingName (actual time=0.597..0.597 rows=1 loops=1)
|   -> Table scan on <temporary> (actual time=0.583..0.583 rows=1 loops=1)
|     -> Aggregate using temporary table (actual time=0.582..0.582 rows=1 loops=1)
|       -> Filter: (hr.RoomNumber is null) (cost=16.6 rows=74.2) (actual time=0.104..0.559 rows=4 loops=1)
|         -> Nested loop antijoin (cost=16.6 rows=74.2) (actual time=0.103..0.558 rows=4 loops=1)
|           -> Nested loop inner join (cost=1.63 rows=6.52) (actual time=0.0409..0.0571 rows=28 loops=1)
|             -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.0312..0.0326 rows=1 loops=1)
|               -> Covering index lookup on f using PRIMARY (UID=751) (cost=0.35 rows=1) (actual time=0.0158..0.0168 rows=1 loops=1)
|               -> Single-row index lookup on b using PRIMARY (BuildingId=f.BuildingId) (cost=0.35 rows=1) (actual time=0.0137..0.0138 rows=1 loops=1)
|             -> Covering index lookup on r using idx_rooms_buid_rmb (BuildingId=f.BuildingId) (cost=0.926 rows=6.52) (actual time=0.0113..0.022 rows=28 loops=1)
|           -> Filter: ((cast(str_to_date(hr.StartTime,'%I:%i %p') as time) <= '12:00:00') and (cast(str_to_date(hr.EndTime,'%I:%i %p') as time) > '12:00:00')) (cost=1.33 rows=11.4) (actual time=0.0177..0.0177 rows=0.857 loops=28)
|         -> Covering index lookup on hr using idx_hardreservations_time (RoomNumber=r.RoomNumber, BuildingId=f.BuildingId) (cost=1.33 rows=11.4) (actual time=0.0115..0.0132 rows=9.36 loops=28)
```

idx_buildname:

```
| -> Sort: AvailableRooms DESC, b.BuildingName (actual time=0.503..0.503 rows=1 loops=1)
|   -> Table scan on <temporary> (actual time=0.492..0.493 rows=1 loops=1)
|     -> Aggregate using temporary table (actual time=0.492..0.492 rows=1 loops=1)
|       -> Filter: (hr.RoomNumber is null) (cost=16.6 rows=74.2) (actual time=0.0977..0.469 rows=4 loops=1)
|         -> Nested loop antijoin (cost=16.6 rows=74.2) (actual time=0.0952..0.465 rows=4 loops=1)
|           -> Nested loop inner join (cost=1.63 rows=6.52) (actual time=0.042..0.0555 rows=28 loops=1)
|             -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.031..0.0321 rows=1 loops=1)
|               -> Covering index lookup on f using PRIMARY (UID=751) (cost=0.35 rows=1) (actual time=0.0158..0.0166 rows=1 loops=1)
|               -> Single-row index lookup on b using PRIMARY (BuildingId=f.BuildingId) (cost=0.35 rows=1) (actual time=0.0128..0.0128 rows=1 loops=1)
|             -> Covering index lookup on r using idx_rooms_buid_rmb (BuildingId=f.BuildingId) (cost=0.926 rows=6.52) (actual time=0.0105..0.0211 rows=28 loops=1)
|           -> Filter: ((cast(str_to_date(hr.StartTime,'%I:%i %p') as time) <= '12:00:00') and (cast(str_to_date(hr.EndTime,'%I:%i %p') as time) > '12:00:00')) (cost=1.33 rows=11.4) (actual time=0.0146..0.0146 rows=0.857 loops=28)
|         -> Covering index lookup on hr using idx_hardreservations_time (RoomNumber=r.RoomNumber, BuildingId=f.BuildingId) (cost=1.33 rows=11.4) (actual time=0.00848..0.0102 rows=9.36 loops=28)
```

The final design we selected is to use the idx_hardreservations_time_room index on the HardReservations (RoomNumber, BuildingId, StartTime, EndTime) attributes. However, from our testing we've seen that indexing the StartTime and EndTime attributes has no effect on the query performance (idx_hardreservations_time), so this index can likely just be reduced to only

index RoomNumber and BuildingId. This index design resulted in a 41.5% improvement in the cost of our query. This performance increase is due to the indexing of HardReservation keys that allow a more efficient join. Since we possibly do 3 joins before the HardReservation join (depending on the join plan that MySQL uses), we join a large number of rows with it. Indexing the start and end times has less of an effect on performance since they're only used in a filtering before the join. Additionally, I expected the index on BuildingName to improve performance since it's used by GROUP BY, however due to the small cardinality of the result table (1 row), the performance improvement wasn't able to be seen. For a user with a larger number of favorite buildings, there may be a performance improvement.

Query 2

This query identifies the top 15 users who have made the most reservations in the North Campus (north of Green St).

```
SELECT U.FullName, U.Email, COUNT(UR.ReservationId) AS
TotalReservations
FROM Users U
JOIN UserReservations UR ON U.UID = UR.UID
JOIN Buildings B ON B.BuildingId = UR.BuildingId
WHERE B.Latitude > 40.1104
GROUP BY U.UID
ORDER BY TotalReservations DESC
LIMIT 15;
```

```
mysql> SELECT U.FullName, U.Email, COUNT(UR.ReservationId) AS TotalReservations
-> FROM Users U
-> JOIN UserReservations UR ON U.UID = UR.UID
-> JOIN Buildings B ON B.BuildingId = UR.BuildingId
-> WHERE B.Latitude > 40.1104
-> GROUP BY U.UID
-> ORDER BY TotalReservations DESC
-> LIMIT 15;
```

FullName	Email	TotalReservations
Julia Miller	jmilller4@illinois.edu	2
Lori Hudson	lhudson7@illinois.edu	2
Clayton Carlson	cc15@illinois.edu	2
Michael Austin	maustin9@illinois.edu	2
Steven Newman	snewman4@illinois.edu	2
Kenneth Hughes	khughes8@illinois.edu	2
Tyler Mullins	tm55@illinois.edu	2
Kevin Gay	kgay6@illinois.edu	2
Thomas Reed	treed4@illinois.edu	2
Andrew Murphy	amurphy8@illinois.edu	2
Brittany Mullen	bmullen6@illinois.edu	2
Allison Cox	acox6@illinois.edu	2
Kristina Whitney	kw21@illinois.edu	2
Heather Fritz	hfritz3@illinois.edu	2
Charles Wright	cwright3@illinois.edu	2

```
15 rows in set (0.03 sec)
```

Index	Columns indexed	Query Cost	Notes
Default	Only PRIMARY and FKs	560	PRIMARY and FK indices already give high speedup
	<pre> -> Sort: TotalReservations DESC (actual time=3.31..3.34 rows=494 loops=1) -> Table scan on <temporary> (actual time=3.01..3.07 rows=494 loops=1) -> Aggregate using temporary table (actual time=3.01..3.01 rows=494 loops=1) -> Nested loop inner join (cost=560 rows=328) (actual time=0.845..2.68 rows=494 loops=1) -> Nested loop inner join (cost=445 rows=328) (actual time=0.827..2.17 rows=494 loops=1) -> Table scan on UR (cost=100 rows=985) (actual time=0.0486..0.124 rows=985 loops=1) -> Filter: (B.Latitude > 40.11040000) (cost=0.25 rows=0.333) (actual time=0.00171..0.00175 rows=0.502 loops=985) -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId) (cost=0.25 rows=1) (actual time=700e-6..758e-6 rows=1 loops=985) -> Single-row index lookup on U using PRIMARY (UID=UR.UID) (cost=0.25 rows=1) (actual time=843e-6..872e-6 rows=1 loops=494) </pre>		
idx_ur_uid ON UserReservations(UID);	Default + UserReservations.UID	560	Reduced runtime by 28% by speeding up U.UID = UR.UID matching. But no help on cost.
	<pre> -> Sort: TotalReservations DESC (actual time=2.39..2.41 rows=494 loops=1) -> Table scan on <temporary> (actual time=2.28..2.33 rows=494 loops=1) -> Aggregate using temporary table (actual time=2.28..2.28 rows=494 loops=1) -> Nested loop inner join (cost=560 rows=328) (actual time=0.0997..1.93 rows=494 loops=1) -> Nested loop inner join (cost=445 rows=328) (actual time=0.0945..1.46 rows=494 loops=1) -> Table scan on UR (cost=100 rows=985) (actual time=0.0756..0.243 rows=985 loops=1) -> Filter: (B.Latitude > 40.11040000) (cost=0.25 rows=0.333) (actual time=961e-6..0.001 rows=0.502 loops=985) -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId) (cost=0.25 rows=1) (actual time=777e-6..806e-6 rows=1 loops=985) -> Single-row index lookup on U using PRIMARY (UID=UR.UID) (cost=0.25 rows=1) (actual time=777e-6..806e-6 rows=1 loops=494) </pre>		
idx_buildings_latitude ON Buildings(Latitude);	Default + UserReservations.UID + Buildings.Latitude	576	Realized that MySQL did not use this index at all. Latitude is used for filtering, not index-based lookup.
	<pre> -> Sort: TotalReservations DESC (actual time=2.59..2.61 rows=494 loops=1) -> Table scan on <temporary> (actual time=2.46..2.52 rows=494 loops=1) -> Aggregate using temporary table (actual time=2.46..2.46 rows=494 loops=1) -> Nested loop inner join (cost=576 rows=374) (actual time=0.0874..2.11 rows=494 loops=1) -> Nested loop inner join (cost=445 rows=374) (actual time=0.0801..1.43 rows=494 loops=1) -> Table scan on UR (cost=100 rows=985) (actual time=0.0431..0.109 rows=985 loops=1) -> Filter: (B.Latitude > 40.11040000) (cost=0.25 rows=0.38) (actual time=957e-6..0.001 rows=0.502 loops=985) -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId) (cost=0.25 rows=1) (actual time=749e-6..778e-6 rows=1 loops=985) -> Single-row index lookup on U using PRIMARY (UID=UR.UID) (cost=0.25 rows=1) (actual time=0.00119..0.00122 rows=1 loops=494) </pre>		
idx_ur_building ON UserReservations(BuildingId);	Default + UserReservations.UID + UserReservations.BuildingId	306	MySQL preferred the BuildingId index over the UID index because there are fewer buildings than users. This reduced cost by 47%.
	<pre> -> Sort: TotalReservations DESC (actual time=2.15..2.18 rows=494 loops=1) -> Table scan on <temporary> (actual time=2.01..2.07 rows=494 loops=1) -> Aggregate using temporary table (actual time=2.01..2.01 rows=494 loops=1) -> Nested loop inner join (cost=306 rows=480) (actual time=0.182..1.64 rows=494 loops=1) -> Nested loop inner join (cost=138 rows=480) (actual time=0.171..1.06 rows=494 loops=1) -> Filter: (B.Latitude > 40.11040000) (cost=4.08 rows=19) (actual time=0.0812..0.131 rows=19 loops=1) -> Covering index range scan on B using idx_buildings_latitude over (40.11040000 < Latitude) (cost=4.08 rows=19) (actual time=0.0636..0.109 rows=19 loops=1) -> Index lookup on UR using idx_ur_building (BuildingId=B.BuildingId) (cost=4.43 rows=25.3) (actual time=0.0214..0.046 rows=26 loops=19) -> Single-row index lookup on U using PRIMARY (UID=UR.UID) (cost=0.25 rows=1) (actual time=989e-6..0.00102 rows=1 loops=494) </pre>		

The final index design we chose is to index the column **BuildingId** of the relation **UserReservations**. Initially we expected indexing the column of **UID** of **UserReservations** can speed up the joining of **Users** and **UserReservations** on **U.UID = UR.UID**; however, it only accelerated runtime by 28% but did not reduce the estimated cost. Later, we noticed a **WHERE** condition on the column **Latitude** of the **Buildings** relation, so we thought indexing **Latitude** can reduce the cost—but cost did not decrease but even increased because filtering did not even utilize the index. However, once we indexed the **BuildingId** column of the **UserReservations** relation, MySQL reordered the joins to replace a table scan on

UserReservations by an index lookup, which dramatically improved query performance, reducing the cost by 47% (from 560 to 306).

Query 3

This query identifies the rooms that have been reserved the most in the past month.

```
SELECT R.RoomNumber, B.BuildingName, COUNT(UR.ReservationId) AS
TotalReservations
FROM UserReservations UR
JOIN Rooms R ON UR.RoomNumber = R.RoomNumber AND UR.BuildingId =
R.BuildingId
JOIN Buildings B ON R.BuildingId = B.BuildingId
WHERE STR_TO_DATE(UR.Date, '%Y-%m-%d') >= CURDATE() - INTERVAL 30 DAY
GROUP BY R.RoomNumber, B.BuildingName
ORDER BY TotalReservations DESC
LIMIT 15; -- Get the top 10 most booked rooms
```

Default Query Result:

```
mysql> SELECT R.RoomNumber, B.BuildingName, COUNT(UR.ReservationId) AS TotalReservations
-> FROM UserReservations UR
-> JOIN Rooms R ON UR.RoomNumber = R.RoomNumber AND UR.BuildingId = R.BuildingId
-> JOIN Buildings B ON R.BuildingId = B.BuildingId
-> WHERE STR_TO_DATE(UR.Date, '%Y-%m-%d') >= CURDATE() - INTERVAL 30 DAY
-> GROUP BY R.RoomNumber, B.BuildingName
-> ORDER BY TotalReservations DESC
-> LIMIT 15;

+-----+-----+-----+
| RoomNumber | BuildingName | TotalReservations |
+-----+-----+-----+
| 1057 | Lincoln Hall | 3 |
| 1022 | Literatures, Cultures & Linguistics Building | 3 |
| 100 | Noyes Laboratory | 3 |
| 4031 | Campus Instructional Facility | 3 |
| G30 | Literatures, Cultures & Linguistics Building | 2 |
| 119 | English Building | 2 |
| 164 | Noyes Laboratory | 2 |
| 2100 | Sidney Lu Mechanical Engineering Building | 2 |
| 429 | Armory | 2 |
| 163 | Noyes Laboratory | 2 |
| 125 | English Building | 2 |
| 2039 | Campus Instructional Facility | 2 |
| 207 | Psychology Building | 2 |
| 116 | Roger Adams Laboratory | 2 |
| 430 | Armory | 2 |
+-----+-----+-----+
15 rows in set (0.03 sec)

mysql> █
```


Default Explain Analyze:

```
| -> Limit: 15 row(s)  (actual time=2.01..2.01 rows=15 loops=1)
      -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk  (actual time=2..2.01 rows=15 loops=1)
            -> Table scan on <temporary>  (actual time=1.92..1.96 rows=119 loops=1)
                  -> Aggregate using temporary table  (actual time=1.92..1.92 rows=119 loops=1)
                        -> Nested loop inner join  (cost=1578 rows=1970) (actual time=0.118..1.71 rows=151 loops=1)
                              -> Nested loop inner join  (cost=889 rows=1970) (actual time=0.108..1.46 rows=151 loops=1)
                                    -> Filter: (str_to_date(UR.`Date`,`%Y-%m-%d`) >= <cache>((curdate() - interval 30 day)))  (cost=200 rows=1970) (actual time=0.0856..1.19 rows=151 loops=1)
                                          -> Table scan on UR  (cost=200 rows=1970) (actual time=0.0609..0.774 rows=1970 loops=1)
                                                -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00161..0.00163 rows=1 loops=1)
                                                    -> Single-row covering index lookup on R using PRIMARY (RoomNumber=UR.RoomNumber, BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00145..0.00148 rows=1 loops=151)
```

Index 1: idx_ur_date ON UserReservations(Date);

```
| -> Limit: 15 row(s)  (actual time=4.7..4.7 rows=15 loops=1)
      -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk  (actual time=4.7..4.7 rows=15 loops=1)
            -> Table scan on <temporary>  (actual time=3.73..3.75 rows=119 loops=1)
                  -> Aggregate using temporary table  (actual time=3.73..3.73 rows=119 loops=1)
                        -> Nested loop inner join  (cost=1578 rows=1970) (actual time=1.56..3.43 rows=151 loops=1)
                              -> Nested loop inner join  (cost=889 rows=1970) (actual time=1.55..3.13 rows=151 loops=1)
                                    -> Filter: (str_to_date(UR.`Date`,`%Y-%m-%d`) >= <cache>((curdate() - interval 30 day)))  (cost=200 rows=1970) (actual time=1.5..2.86 rows=151 loops=1)
                                          -> Table scan on UR  (cost=200 rows=1970) (actual time=0.309..1.18 rows=1970 loops=1)
                                                -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00152..0.00156 rows=1 loops=151)
                                                    -> Single-row covering index lookup on R using PRIMARY (RoomNumber=UR.RoomNumber, BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.0018..0.00183 rows=1 loops=151)
```

Index 2: idx_ur_rn ON UserReservations(RoomNumber);

```
| -> Limit: 15 row(s)  (actual time=4.29..4.29 rows=15 loops=1)
      -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk  (actual time=4.29..4.29 rows=15 loops=1)
            -> Table scan on <temporary>  (actual time=4.18..4.18 rows=119 loops=1)
                  -> Aggregate using temporary table  (actual time=4.18..4.18 rows=119 loops=1)
                        -> Nested loop inner join  (cost=1578 rows=1970) (actual time=1.95..3.88 rows=151 loops=1)
                              -> Nested loop inner join  (cost=889 rows=1970) (actual time=1.93..3.43 rows=151 loops=1)
                                    -> Filter: (str_to_date(UR.`Date`,`%Y-%m-%d`) >= <cache>((curdate() - interval 30 day)))  (cost=200 rows=1970) (actual time=1.89..3.14 rows=151 loops=1)
                                          -> Table scan on UR  (cost=200 rows=1970) (actual time=0.081..0.872 rows=1970 loops=1)
                                                -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00165..0.00168 rows=1 loops=151)
                                                    -> Single-row covering index lookup on R using PRIMARY (RoomNumber=UR.RoomNumber, BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00243..0.00246 rows=1 loops=151)
```

Index 3: idx_ur_buid ON Rooms(BuildingId);

```
| -> Limit: 15 row(s)  (actual time=1.95..1.96 rows=15 loops=1)
      -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk  (actual time=1.95..1.95 rows=15 loops=1)
            -> Table scan on <temporary>  (actual time=1.88..1.89 rows=119 loops=1)
                  -> Aggregate using temporary table  (actual time=1.87..1.87 rows=119 loops=1)
                        -> Nested loop inner join  (cost=1578 rows=1970) (actual time=0.0977..1.67 rows=151 loops=1)
                              -> Nested loop inner join  (cost=889 rows=1970) (actual time=0.0836..1.35 rows=151 loops=1)
                                    -> Filter: (str_to_date(UR.`Date`,`%Y-%m-%d`) >= <cache>((curdate() - interval 30 day)))  (cost=200 rows=1970) (actual time=0.0663..1.14 rows=151 loops=1)
                                          -> Table scan on UR  (cost=200 rows=1970) (actual time=0.0479..0.751 rows=1970 loops=1)
                                                -> Single-row index lookup on B using PRIMARY (BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00115..0.00118 rows=1 loops=151)
                                                    -> Single-row covering index lookup on R using PRIMARY (RoomNumber=UR.RoomNumber, BuildingId=UR.BuildingId)  (cost=0.25 rows=1) (actual time=0.00194..0.00196 rows=1 loops=151)
```

Index	Columns Indexed	Query Cost	Notes
Default	Default	1578	Default behavior. High cost because str_to_date() function is expensive, converting every string to a date object. Combine that with all rows being read and being converted, it is very slow.
idx_ur_date ON UserReservations(Date);	UserReservations ('Date')	1578	Realized that MySQL didn't use this index at all. This is because the column is wrapped in a function. From additional research, I would need a functional index for this.
idx_ur_rn ON UserReservations(RoomNumber);	UserReservations (RoomNumber)	1578	Realized that MySQL didn't use this index at all. This is because both sides of the join statement are needed and so the index for just one column is ignored.
idx_ur_buid ON Rooms(BuildingId);	Rooms(BuildingId)	1578	Realized that MySQL didn't use this index at all. This is because the Primary Key can't be accessed any more efficiently.

Overall, I chose not to use any of the indexes as none of them provided any performance upgrade. Among all the three different indexes, none were properly utilized by MySQL due to the expensive nature of the query. Proper optimizations would involve more complex, functional indexes, and maybe finding a way to not have to do the string and date conversions, which would be better at vastly reducing cost.

Query 4

This query retrieves the buildings with the highest total number of reservations (aggregating both user reservations and hard reservations).

```
mysql> SELECT B.BuildingName, COUNT(AllReservations.ReservationId) AS TotalReservations
-> FROM Buildings B
-> JOIN Rooms R ON B.BuildingId = R.BuildingId
-> JOIN (
->   -- Combine user reservations and hard reservations, keeping RoomNumber
->   SELECT UR.ReservationId, UR.BuildingId, UR.RoomNumber
->   FROM UserReservations UR
->   UNION ALL
->   SELECT H.EventId AS ReservationId, H.BuildingId, H.RoomNumber
->   FROM HardReservations H
-> ) AS AllReservations
-> ON R.BuildingId = AllReservations.BuildingId
-> AND R.RoomNumber = AllReservations.RoomNumber
-> GROUP BY B.BuildingId
-> ORDER BY TotalReservations DESC
-> LIMIT 15; -- Top 15 buildings with the most reservations
```

BuildingName	TotalReservations
Online	949
Literatures, Cultures, & Ling	446
David Kinley Hall	445
Campus Instructional Facility	443
Gregory Hall	431
Armory	385
Lincoln Hall	354
Noyes Laboratory	351
English Building	340
Electrical & Computer Eng Bldg	326
Business Instructional Fac	318
Davenport Hall	253
Education Building	245
Sidney Lu Mech Engr Bldg	239
Wohlers Hall	217

Indexing portion:

Index	Columns indexed	Query Cost	Notes
Default	Only primary and FKs	9559	No indexes beyond default keys. High cost due to full scans and expensive joins.
idx_rooms_capacity	Rooms(Capacity)	17290	Significantly increased the query cost. The optimizer may have

			chosen a less efficient access path, possibly due to low selectivity or because this column is not used in joins or filtering.
idx_hr_buid_rnb	HardReservations(BuildingId, RoomNumber)	9441	Minor cost drop. Optimized access to HardReservations subquery using indexed join keys.
idx_ur_roomonly	UserReservations(RoomNumber)	9559	No improvement over default. The index was likely ignored since the join condition includes both BuildingId and RoomNumber, making this partial index less useful.

Default Explain Analyze:

```
| -> Limit: 15 row(s) (actual time=26.8..26.8 rows=15 loops=1)
  -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk (actual time=26.8..26.8 rows=15 loops=1)
    -> Table scan on <temporary> (actual time=26.7..26.8 rows=117 loops=1)
      -> Aggregate using temporary table (actual time=26.7..26.7 rows=117 loops=1)
        -> Nested loop inner join (cost=9559 rows=10298) (actual time=6.22..19.8 rows=10795 loops=1)
          -> Nested loop inner join (cost=5855 rows=10298) (actual time=6.17..13.1 rows=10795 loops=1)
            -> Table scan on AllReservations (cost=2219..2350 rows=10298) (actual time=6.14..7.82 rows=10795 loops=1)
              -> Union all materialize (cost=2219..2219 rows=10298) (actual time=6.14..6.14 rows=10795 loops=1)
                -> Covering index scan on UR using RoomNumber (cost=200 rows=1970) (actual time=0.101..0.661 rows=1970 loops=1)
                  -> Covering index scan on H using RoomNumber (cost=990 rows=8328) (actual time=0.055..2.44 rows=8825 loops=1)
                -> Single-row index lookup on B using PRIMARY (BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=290e-6..320e-6 rows=1 loops=10795)
                -> Single-row covering index lookup on R using PRIMARY (RoomNumber=AllReservations.RoomNumber, BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=418e-6..446e-6 rows=1 loops=10795)
```

CREATE INDEX idx_rooms_capacity ON Rooms(Capacity);

Explain Analyze:

```
| -> Limit: 15 row(s) (actual time=62.5..62.5 rows=15 loops=1)
  -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk (actual time=62.5..62.5 rows=15 loops=1)
    -> Table scan on <temporary> (actual time=62.4..62.4 rows=117 loops=1)
      -> Aggregate using temporary table (actual time=62.4..62.4 rows=117 loops=1)
        -> Nested loop inner join (cost=17290 rows=10298) (actual time=40.2..55.3 rows=10795 loops=1)
          -> Nested loop inner join (cost=13686 rows=10298) (actual time=40.1..47.5 rows=10795 loops=1)
            -> Table scan on AllReservations (cost=2227..2358 rows=10298) (actual time=38.5..40.1 rows=10795 loops=1)
              -> Union all materialize (cost=2227..2227 rows=10298) (actual time=38.5..38.5 rows=10795 loops=1)
                -> Covering index scan on UR using RoomNumber (cost=207 rows=1970) (actual time=2.14..5.02 rows=1970 loops=1)
                  -> Covering index scan on H using idx_hardreservations_time (cost=990 rows=8328) (actual time=2.43..28.7 rows=8825 loops=1)
                -> Single-row index lookup on B using PRIMARY (BuildingId=AllReservations.BuildingId) (cost=1 rows=1) (actual time=482e-6..512e-6 rows=1 loops=10795)
                -> Single-row covering index lookup on R using PRIMARY (RoomNumber=AllReservations.RoomNumber, BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=477e-6..506e-6 rows=1 loops=10795)
```

CREATE INDEX idx_hr_buid_rnb ON HardReservations(BuildingId, RoomNumber);

Explain Analyze:

```
| -> Limit: 15 row(s) (actual time=27..27 rows=15 loops=1)
  -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk (actual time=27..27 rows=15 loops=1)
    -> Table scan on <temporary> (actual time=26.8..26.9 rows=117 loops=1)
      -> Aggregate using temporary table (actual time=26.8..26.8 rows=117 loops=1)
        -> Nested loop inner join (cost=9441 rows=10298) (actual time=6.83..20.1 rows=10795 loops=1)
          -> Nested loop inner join (cost=5837 rows=10298) (actual time=6.82..13.7 rows=10795 loops=1)
            -> Table scan on AllReservations (cost=2101..2233 rows=10298) (actual time=6.77..8.35 rows=10795 loops=1)
              -> Union all materialize (cost=2101..2101 rows=10298) (actual time=6.77..6.77 rows=10795 loops=1)
                -> Covering index scan on UR using RoomNumber (cost=200 rows=1970) (actual time=0.0815..0.611 rows=1970 loops=1)
                  -> Covering index scan on H using RoomNumber (cost=872 rows=8328) (actual time=0.0472..2.68 rows=8825 loops=1)
                -> Single-row index lookup on B using PRIMARY (BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=285e-6..320e-6 rows=1 loops=10795)
                -> Single-row covering index lookup on R using PRIMARY (RoomNumber=AllReservations.RoomNumber, BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=400e-6..429e-6 rows=1 loops=10795)
```

CREATE INDEX idx_ur_roomonly ON UserReservations(RoomNumber);

Explain Analyze:

```

| -> Limit: 15 row(s) (actual time=27.6..27.6 rows=15 loops=1)
    -> Sort: TotalReservations DESC, limit input to 15 row(s) per chunk (actual time=27.6..27.6 rows=15 loops=1)
        -> Table scan on <temporary> (actual time=27.6..27.6 rows=117 loops=1)
            -> Aggregate using temporary table (actual time=27.6..27.6 rows=117 loops=1)
                -> Nested loop inner join (cost=9559 rows=10298) (actual time=6.42..20.7 rows=10795 loops=1)
                    -> Table scan on AllReservations (cost=2219..2350 rows=10298) (actual time=6.39..8.12 rows=10795 loops=1)
                        -> Union all materialize (cost=2219..2219 rows=10298) (actual time=6.38..6.38 rows=10795 loops=1)
                            -> Covering index scan on UR using RoomNumber (cost=200 rows=1970) (actual time=0.056..0.592 rows=1970 loops=1)
                                -> Covering index scan on H using idx_hardreservations_time (cost=990 rows=8328) (actual time=0.0436..2.65 rows=8825 loops=1)
                                    -> Single-row index lookup on B using PRIMARY (BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=311e-6..340e-6 rows=1 loops=10795)
                                        -> Single-row covering index lookup on R using PRIMARY (RoomNumber=AllReservations.RoomNumber, BuildingId=AllReservations.BuildingId) (cost=0.25 rows=1) (actual time=435e-6..464e-6 rows=1 loops=10795)

```

The final index design I selected is `idx_hr_buid_rnb` on the `HardReservations` table, indexing the composite key (`BuildingId`, `RoomNumber`). This index was chosen because it directly supports the join condition between the derived `AllReservations` table and `Rooms`, which compares both `BuildingId` and `RoomNumber`. Among all the valid indexing strategies tested, this one provided the only measurable improvement, reducing the query cost from 9559 to 9441. Other indexes, such as those on `Rooms(Capacity)` and `UserReservations(RoomNumber)`, either increased the cost or had no effect. While the performance gain was modest, it demonstrates that indexing join keys in tables contributing to large UNION operations can assist the optimizer in generating more efficient execution plans.